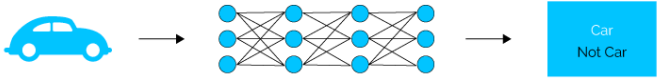
Optimization for Deep Learning (OPT4DL)

Abolfazl Hashemi, Purdue ECE

Machine Learning (ML) Basics

Machine Learning: Learning from Data

- We have $\mathcal{D} = \{z_1, \ldots, x_n\}$ a collection of n training data points/samples z_i's
- Think of z_i as a collection of features and their corresponding label, that is $z_i = (x_i, y_i)$
  - $x_i \in \mathbb{R}^p$ is a vector of p features, e.g., an image with p pixels
  - $y_i \in \mathbb{R}$ is the label for the ith sample, e.g., whether the image is a car or not



- Typically, ML problems are one of the followings
  - Supervised Learning
  - Unsupervised Learning
  - Reinforcement Learning



- Assume there exists some true but unknown rule between features and labels $y_i = A^*(x_i)$ for all i
- Choose a parametric model $A(\cdot, w)$ such that for some w^*, $A(\cdot, w^*) \approx A^*(\cdot)$
- Training: fit the model to $\mathcal{D}$ to find w^*
- Let $\ell(w, z_i)$ denote the loss/cost of a model w fitting sample i
  - Example: quadratic loss $\ell(w, z_i) = \frac{1}{2}(A(x_i, w) - y_i)^2$
- Training as an optimization problem

$$\min_{w \in \mathbb{R}^d} f(w) = \frac{1}{n} \sum_{i=1}^{n} \ell(w, z_i) \tag{1}$$

- That is, minimize the average loss over the training data
- Let w^* denote the optimal solution

Performance on Unseen Data



- If training successful, model will fit training data well
- To assess performance on new, unseen data (data $\notin \mathcal{D}$) we make an assumption:
  - New and training data belong to the same universe
  - Let z be a new data, then $z \sim p_z$ and $\mathcal{D} \sim p_z$ for some data distribution p_z
- Measure of performance

$$\min_{w \in \mathbb{R}^d} \bar{f}(w) = \mathbb{E}_{z \sim p_z}[\ell(w, z_i)] \tag{2}$$

  how well we perform on average over the entire universe (not just over the training data)
- Note that f and $\bar{f}$ are not necessarily the same, f an empirical approximation of $\bar{f}$

Example: A Discrete Universe



- Let p_z be discrete and $p_z(Z = z_i) = \frac{1}{n}$
- That is, our data universe is discrete and each data is equally likely to be observed
- Then, f and $\bar{f}$ are



- To ensure performance on all data (both seen and unseen), we aim to minimize the Population Loss/Risk

$$\bar{w} := \arg\min_{w \in \mathbb{R}^d} \bar{f}(w) = \mathbb{E}_{z \sim p_z}[\ell(w, z_i)] \tag{3}$$

- But, all we can do in practice is to minimize the Empirical Loss/Risk

$$w^* := \arg\min_{w \in \mathbb{R}^d} f(w) = \frac{1}{n} \sum_{i=1}^{n} \ell(w, z_i) \tag{4}$$

  as p_z is unknown

- This approach is called Empirical Risk Minimization (ERM)

How good is Approximately Solving ERM?



- Let $\hat{w}$ be an approximate/suboptimal solution to ERM
- And recall $\bar{w}$ and w^* are the minimizers of Population loss ($\bar{f}$) and Empirical loss (f), respectively.
- We need to bound the Estimation Error: $\bar{f}(\hat{w}) - \bar{f}(\bar{w})$

$$\begin{aligned}\bar{f}(\hat{w}) - \bar{f}(\bar{w}) &= \bar{f}(\hat{w}) \pm f(\hat{w}) \pm f(w^*) \pm f(\bar{w}) - \bar{f}(\bar{w}) \\ &= f(\hat{w}) - f(w^*) \\ &+ \bar{f}(\hat{w}) - f(\hat{w}) \\ &+ f(w^*) - \bar{f}(w^*) \\ &+ f(w^*) - f(\bar{w})\end{aligned} \tag{5}$$

- estimation error $\leq$ optimization error $+2\times$ empirical approximation error
- Controlled by better training algorithms and better/larger datasets

Representation Error



- Recall $\bar{w}$ is the minimizer of Population loss ($\bar{f}$)
- Recall we assume $y = A^*(x)$
- If our model is not powerful enough, even by minimizing the Population Loss, we may not fully learn the feature-label relations
- Mathematically, $A^*(\cdot)$ and $A(\cdot, \bar{w})$ are not necessarily the same
- Representation error $= \bar{f}(\bar{w}) - f_{opt}$ where f_{opt} is the loss if we knew the true relation (we can assume $f_{opt} \approx 0$)

- Typically, representation error decreases as DL models become deeper and/or wider (more powerful)
- Total Learning Error = Representation Error + Optimization/Training Error + Empirical Approximation Error

Examples of Models



- Binary Classification
  - Features: $x \in \mathbb{R}^d$
  - Labels: $y \in \{-1, +1\}$
  - Linearly separable classes: $\hat{y} = \text{sign}(x^\top w)$
  - 0-1 loss: $\ell(w, z) = \mathbb{I}[y \neq \text{sign}(x^\top w)]$
  - Hinge loss: $\ell(w, z) = \max(0, 1 - x^\top w)$
  - ERM with hinge becomes a convex task
  - The linear relation $x^\top w$ could be replaced by a DL model $A(x, w)$, resulting in a nonconvex training tasks

Examples of Models (Cont'd)



- Linear Regression
  - Features: $x \in \mathbb{R}^d$
  - Labels: $y \in \mathbb{R}$
  - Linear dependency: $y = x^\top w + e$ where $e \sim \mathcal{N}(0, \sigma^2)$
  - Square loss: $\ell(w, z) = \frac{1}{2}(y - x^\top w)^2$
  - ERM becomes a convex task
  - The linear relation $x^\top w$ could be replaced by a DL model $A(x, w)$, resulting in a nonconvex training task

Examples of Models (Cont'd)



- Logistic Regression
  - Features: $x \in \mathbb{R}^d$
  - Labels: $y \in \{0, 1\}$
  - Sigmoid/logistic activation:

$$y = \frac{1}{1 + \exp(-x^\top w)} \tag{6}$$

  - $\frac{1}{1+\exp(-x^\top w)}$ is thought as the probability that the label of x is 1.
  - Binary Cross-entropy loss:

$$\ell(w, z) = -y \log \frac{1}{1 + \exp(-x^\top w)} - (1 - y) \log(1 - \frac{1}{1 + \exp(-x^\top w)}) \tag{7}$$

  - ERM becomes a convex task
  - The linear relation $x^\top w$ could be replaced by a DL model $A(x, w)$, resulting in a nonconvex training task